

Lecture 3 - Supervised Learning

Supervised learning is the branch of machine learning in which a model is trained on labelled examples. Each training example is a pair consisting of an input object, usually a vector of features, and a desired output value, called the label. The learning algorithm searches for a function that maps inputs to outputs in a way that generalises to examples it has never seen. Generalisation, rather than memorisation, is the entire point: a model that reproduces its training set perfectly but fails on new data has learned nothing useful.

The two dominant families of supervised task are classification and regression. In classification the label is drawn from a finite set of categories, such as deciding whether an email is spam or not spam, or which of ten digits a handwritten image shows. In regression the label is continuous, such as predicting the price of a house from its size, location and age. The distinction matters because it determines which loss function is appropriate. Classification typically uses cross-entropy loss, while regression typically uses mean squared error or mean absolute error.

Training proceeds by minimising the loss over the training set, usually with gradient descent or one of its variants. The gradient of the loss with respect to each parameter tells the optimiser which direction reduces the error, and the learning rate controls how large a step is taken in that direction. A learning rate that is too large causes the optimiser to overshoot and diverge, while one that is too small makes training impractically slow. Choosing it well is one of the most consequential decisions in the whole training pipeline.

Lecture 4 - Overfitting, Regularisation and Validation

A model overfits when it captures noise in the training data as though it were signal. The symptom is a widening gap between training error and validation error: training error keeps falling while validation error flattens and then rises. Overfitting is not a bug in the algorithm, it is a consequence of having more capacity than the data can support, and it becomes more likely as the number of parameters grows relative to the number of training examples.

Regularisation is the standard family of remedies. L2 regularisation, also called weight decay, adds the squared magnitude of the weights to the loss, which pulls the solution towards smaller weights and smoother functions. L1 regularisation adds the absolute magnitude instead, which tends to drive some weights exactly to zero and so performs a kind of automatic feature selection. Dropout, used in neural networks, randomly disables a fraction of units during each training step, forcing the network not to rely on any single unit. Early stopping simply halts training at the point where validation error stops improving.

None of these techniques can be tuned honestly using the test set. The standard protocol splits the data three ways: a training set to fit parameters, a validation set to choose hyperparameters such as the regularisation strength, and a test set that is looked at once, at the very end, to estimate generalisation error. When data is scarce, k-fold cross-validation reuses the data more efficiently by rotating which fold serves as validation, at the cost of training the model k times.

Lecture 5 - Evaluation Metrics

Accuracy, the fraction of predictions that are correct, is the most intuitive metric and often the most misleading. On an imbalanced dataset where ninety-nine per cent of examples belong to one class, a model that always predicts the majority class achieves ninety-nine per cent accuracy while being completely useless. This is why the confusion matrix, which counts true positives, false positives, true negatives and false negatives separately, is the honest starting point for any evaluation.

Precision is the fraction of predicted positives that are genuinely positive, and it answers the question: when the model raises an alarm, how often is it right. Recall is the fraction of genuine positives that the model successfully found, and it answers: of everything that should have been caught, how much was. The two trade off against each other as the decision threshold moves, and the F1 score, the harmonic mean of precision and recall, summarises that trade-off in a single number. The harmonic mean is used rather than the arithmetic mean because it punishes a large imbalance between the two.

Which metric to optimise is a question about consequences, not mathematics. A medical screening test that misses a disease is far more costly than one that raises a false alarm, so recall dominates. A spam filter that discards a legitimate message is worse than one that lets some spam through, so precision dominates. Reporting a single aggregate figure without stating which errors it treats as acceptable hides exactly the information a reader needs.

Lecture 6 - Feature Engineering and Data Leakage

Feature engineering is the process of turning raw observations into inputs a model can use. Numerical features are often standardised so that each has zero mean and unit variance, which stops features measured on large scales from dominating distance-based methods and speeds up gradient descent. Categorical features must be encoded numerically, most commonly with one-hot encoding, which creates one binary column per category, or with target encoding, which replaces each category by a statistic of the label within that category.

Data leakage is the most damaging and least visible failure in applied machine learning. It occurs whenever information that would not be available at prediction time leaks into the training features. A classic case is standardising the whole dataset before splitting it, so the training set has absorbed the mean and variance of the test set. Another is including a feature that is a downstream consequence of the label rather than a cause of it, such as using the number of treatments a patient received to predict whether they were diagnosed.

Leakage announces itself as results that are too good to be true: near-perfect validation scores that collapse the moment the model meets genuinely new data. The defence is procedural rather than algorithmic. Fit every transformation on the training fold only, apply it to the validation fold, and be able to say for every feature exactly when its value becomes known relative to the moment of prediction.

Lecture 7 - The Bias-Variance Decomposition

The expected error of a model on unseen data decomposes into three parts: bias, variance, and irreducible noise. Bias measures how far the average prediction of the model, taken over many possible training sets, is from the true value. It reflects assumptions the model class makes that the data does not satisfy. A linear model applied to a curved relationship has high bias no matter how much data it is given.

Variance measures how much the model's prediction changes when it is retrained on a different sample from the same distribution. A very flexible model, such as an unpruned decision tree, can fit almost any training set closely, which means its predictions depend heavily on the particular sample it saw. Irreducible noise is the part of the target that no model can explain because it is genuinely random or depends on variables that were never measured.

The practical consequence is that model complexity is a trade-off rather than a quantity to maximise. Increasing complexity reduces bias and raises variance; decreasing it does the reverse. Total error falls and then rises as complexity grows, tracing the familiar U-shaped curve, and the useful model sits near its minimum. Ensemble methods work by attacking one term at a time: bagging averages many high-variance models to cut variance, while boosting combines many high-bias models to cut bias.

Lecture 8 - Gradient Descent Variants

Batch gradient descent computes the gradient of the loss over the entire training set before taking a single step. The direction it takes is the true gradient, so the path is smooth, but every step requires a full pass over the data, which is prohibitive when the dataset is large. Stochastic gradient descent goes to the other extreme and updates the parameters after every single example. Each step is cheap and noisy, and the noise turns out to be useful because it can shake the optimiser out of poor local minima.

Mini-batch gradient descent is the compromise used almost universally in practice. It computes the gradient over a small batch, typically between thirty-two and five hundred and twelve examples, which is large enough for the gradient estimate to be reasonably stable and small enough to fit in memory and exploit vectorised hardware. Batch size interacts with learning rate: larger batches give less noisy gradients and tolerate larger learning rates.

Momentum accelerates convergence by accumulating an exponentially decaying average of past gradients, which damps oscillation across narrow valleys and builds speed along consistent directions. Adam goes further by keeping running averages of both the gradient and its square, giving every parameter its own effective learning rate. Adam usually converges fastest with little tuning, which is why it is the common default, though well-tuned momentum sometimes generalises slightly better.